

```
ParentClass objects don't have this method.  
Create a GrandchildClass object and call its methods:  
Hello, world!  
ParentClass objects don't have this method.  
Only GrandchildClass objects have this method.  
An error:  
Traceback (most recent call last):  
  File "inheritanceExample.py", line 35, in <module>  
    parent.someNewMethod() # ParentClass objects don't have this method.  
AttributeError: 'ParentClass' object has no attribute 'someNewMethod'
```

We've created three classes named `ParentClass` ❶, `ChildClass` ❸, and `GrandchildClass` ❹. The `ChildClass` *subclasses* `ParentClass`, meaning that `ChildClass` will have all the same methods as `ParentClass`. We say that `ChildClass` *inherits* methods from `ParentClass`. Also, `GrandchildClass` *subclasses* `ChildClass`, so it has all the same methods as `ChildClass` and its parent, `ParentClass`.

Using this technique, we've effectively copied and pasted the code for the `printHello()` method ❷ into the `ChildClass` and `GrandchildClass` classes. Any changes we make to the code in `printHello()` update not only `ParentClass`, but also `ChildClass` and `GrandchildClass`. This is the same as changing the code in a function updates all of its function calls. You can see this relationship in [Figure 16-1](#). Notice that in class diagrams, the arrow is drawn from the subclass pointing to the base class. This reflects the fact that a class will always know its base class but won't know its subclasses.